

1. El Ecosistema: Herramientas y Paradigmas

¿Qué es .NET?

.NET es un **framework o ecosistema de desarrollo de código abierto** creado por Microsoft. Funciona como la "infraestructura o cocina equipada" donde se ejecutan las aplicaciones. Proporciona bibliotecas de clases preescritas, gestión de memoria automática (Garbage Collector) y un entorno seguro para compilar y ejecutar código sin importar si estás en Windows, Linux o macOS.

¿Qué es C#?

C# (pronunciado *C-sharp*) es un **lenguaje de programación moderno, orientado a objetos y fuertemente tipado**. Fue diseñado para aprovechar al máximo el framework .NET. Es el lenguaje estándar para construir desde aplicaciones de consola simples hasta sistemas web empresariales robustos, APIs y videojuegos.

¿Qué es Visual Studio?

Visual Studio es el **Entorno de Desarrollo Integrado (IDE)** oficial y más potente para .NET. No es un simple editor de texto; incluye un depurador avanzado (debugger), autocompletado inteligente (IntelliSense), herramientas de gestión de proyectos, compiladores integrados y soporte nativo para pruebas y despliegue.

¿Qué es la Programación Orientada a Objetos (POO)?

La POO es un **paradigma de programación** que utiliza "objetos" para modelar el mundo real. En lugar de ver el software como una simple lista de instrucciones secuenciales, la POO divide el sistema en piezas modulares (clases y objetos) que interactúan entre sí. Sus pilares principales son la encapsulación, la herencia y el polimorfismo.

2. Estructuras Fundamentales (Sintaxis Básica)

Variables

Una **variable** es un espacio reservado en la memoria RAM para almacenar un valor temporalmente. Como C# es *fuertemente tipado*, cada variable debe declarar explícitamente qué tipo de dato contendrá.

- **Tipos comunes:** int (enteros), decimal (monetarios/precisión), string (textos), bool (verdadero/falso).

C#

```
string nombreCurso = "Desarrollo .NET";  
int horasDuracion = 4;
```

```
decimal precio = 0.00m;  
bool estaActivo = true;
```

Condicionales (if, else, switch)

Permiten que el programa tome decisiones lógicas alterando el flujo de ejecución según se cumplan o no ciertas reglas de negocio.

```
C#  
int puntaje = 85;  
  
if (puntaje >= 90) {  
    Console.WriteLine("Excelente");  
} else if (puntaje >= 70) {  
    Console.WriteLine("Aprobado");  
} else {  
    Console.WriteLine("Reprobado");  
}
```

Ciclos o Bucles (for, foreach, while)

Automatizan la repetición de bloques de código de forma controlada para evitar la duplicación manual.

```
C#  
// Ciclo For (repetición exacta)  
for (int i = 1; i <= 3; i++) {  
    Console.WriteLine($"Iteración número: {i}");  
}  
  
// Ciclo Foreach (recorrido de colecciones)  
string[] tecnologias = { "C#", "SQL", "Angular" };  
foreach (var tech in tecnologias) {  
    Console.WriteLine($"Tecnología: {tech}");  
}
```

Arreglos (Arrays)

Un **arreglo** es una estructura de datos estática que almacena una **cantidad fija** de elementos del mismo tipo en posiciones contiguas de memoria. Su tamaño no puede cambiar una vez creado.

```
C#  
// Arreglo estático de 3 elementos  
int[] calificaciones = new int[3];  
calificaciones[0] = 90;
```

```
calificaciones[1] = 85;  
calificaciones[2] = 95;
```

Listas (List)

A diferencia de los arreglos, una **Lista** es una estructura **dinámica**. Puede crecer, reducirse, ordenar y buscar elementos automáticamente en tiempo de ejecución. Es la estructura más usada en el desarrollo profesional.

```
C#  
using System.Collections.Generic;
```

```
List usuarios = new List();  
usuarios.Add("Carlos");  
usuarios.Add("María");
```

```
// Removiendo un elemento  
usuarios.Remove("Carlos");
```

3. Arquitectura y Componentes Avanzados

Clases y Objetos

- **Clase:** Es el **plano arquitectónico** o molde genérico (por ejemplo, la clase Vehículo). Define qué propiedades y comportamientos tendrán los elementos creados a partir de ella.
- **Objeto:** Es la **instancia real** en memoria generada a partir de esa clase (por ejemplo, un automóvil específico color rojo con placas XYZ).

```
C#  
// 1. Definición de la Clase (El molde)  
public class Empleado {  
    public string Nombre { get; set; }  
    public decimal Salario { get; set; }  
  
    public void MostrarDatos() {  
        Console.WriteLine($"Empleado: {Nombre}, Salario:\{Salario}");  
    }  
}  
  
// 2. Uso en el Main (Creación del Objeto)  
// Empleado emp = new Empleado();  
// emp.Nombre = "Ana";  
// emp.Salario = 15000;  
// emp.MostrarDatos();
```

Modelos (POCOs)

Un **Modelo** (frecuentemente llamado POCO por *Plain Old CLR Object*) es una clase especializada cuya única misión es **transportar y estructurar datos**, representando por lo general las tablas de una base de datos o las estructuras JSON que viajan por una API. No suelen contener lógica de negocio compleja.

C#

```
public class ProductoModel {  
    public int Id { get; set; }  
    public string NombreProducto { get; set; } = string.Empty;  
    public decimal PrecioUnitario { get; set; }  
}
```

Métodos

Un **método** es un bloque de código encapsulado que realiza una tarea específica. Evita repetir código y permite organizar la lógica en partes más pequeñas. Puede recibir parámetros de entrada y retornar un valor mediante return, o usar void si solo ejecuta una acción.

C#

```
public class Operaciones {  
    // Método que recibe parámetros y retorna un resultado numérico  
    public int Multiplicar(int a, int b) {  
        return a * b;  
    }  
  
    // Método tipo void (no retorna valor, solo ejecuta una acción)  
    public void ImprimirAlerta(string mensaje) {  
        Console.WriteLine($"[ALERTA]: {mensaje}");  
    }  
}
```

Interfaces

Una **Interfaz** (interface) actúa estrictamente como un **contrato o plantilla de comportamiento**. Define *qué* métodos y propiedades obligatoriamente debe tener una clase, pero no incluye la lógica de cómo se ejecutan. Es el pilar fundamental para aplicar la Inyección de Dependencias y desacoplar componentes en .NET Core.

C#

```
// Definición del contrato  
public interface IRepository {  
    void Guardar(string entidad);  
    string Obtener(int id);  
}
```

```
}
```

```
// Implementación obligatoria del contrato en una clase concreta  
public class RepositorioSql : IRepository {  
    public void Guardar(string entidad) {  
        Console.WriteLine($"Guardando '{entidad}' en SQL Server...");  
    }  
  
    public string Obtener(int id) {  
        return "Datos recuperados de SQL";  
    }  
}
```